

LAPORAN PRAKTIKUM STRUKTUR DATA

Sorting 1

disusun oleh :

Khairunnisa M.

NIM 2511532005

Dosen Pengampu :

Dr. Wahyudi,. S.T., M.T.

Asisten Dosen :

M. Ghalid



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji Syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan karunia-Nya laporan praktikum Struktur Data dengan materi Sorting ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi dan pemahaman penulis terhadap konsep Sorting dalam struktur data serta implementasinya menggunakan bahasa pemrograman Java.

Dalam laporan ini, penulis membahas beberapa Jenis dan implementasi Sorting , serta operasi – operasi dasar yang terdapat pada Sorting.

Penulis menyadari bahwa lapoaran ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang. Akhir kata, semeoga laporan ini dapat memberikan manfaat bagi pembaca.

Padang, 18 Mei 2026

Khairunnisa M.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I.....	2
PENDAHULUAN	2
1.1. Latar Belakang	2
1.2. Tujuan Praktikum	2
1.3. Manfaat Praktikum.....	2
BAB II	3
PEMBAHASAN	3
2.1. Deskripsi Materi Double Linked List	3
2.2. InsertionSort_2511532005.....	4
2.3. SelectionSort_2511532005.....	5
2.4. BubbleSort_2511532005.....	6
2.5. InsertionGUI_2511532005	7
2.6. Analisis Praktikum.....	9
BAB III.....	11
KESIMPULAN DAN SARAN.....	11
3.1 Kesimpulan.....	11
1.2 Saran.....	11
DAFTAR PUSTAKA	12

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dalam dunia pemrograman dan pengolahan data, proses pengurutan (sorting) merupakan salah satu Teknik dasar yang sangat penting. Pengurutan data digunakan untuk menyusun data berdasarkan urutan tertentu, seperti dari nilai terkecil ke terbesar atau sebaliknya. Data yang terurut akan mempermudah proses pencarian, analisis, dan pengolahan informasi.

Pada mata kuliah Struktur Data, mahasiswa diperkenalkan dengan berbagai algoritma pengurutan sederhana sebagai dasar untuk memahami konsep efisiensi algoritma. Beberapa metode sorting yang umum dipelajari adalah *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*. Ketiga algoritma ini memiliki cara kerja yang berbeda dalam mengurutkan data, namun sama-sama bertujuan menghasilkan susunan data yang teratur.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami langkah kerja masing-masing algoritma sorting, menganalisis proses pengurutan data, serta mengimplementasikannya ke dalam bahasa pemrograman. Pemahaman terhadap algoritma sorting juga menjadi dasar penting untuk mempelajari algoritma yang lebih kompleks pada tahap berikutnya.

1.2. Tujuan Praktikum

1. Memahami konsep dasar sorting pada Struktur Data.
2. Mempelajari cara kerja algoritma Insertion Sort, Selection Sort, dan Bubble Sort.
3. Mengimplementasiakan algoritma sorting menggunakan Bahasa pemrograman Java.

4. Mengetahui perbedaan proses pengurutan pada masing-masing metode sorting.
5. Membuat visualisasi proses sorting menggunakan GUI Java Swing.

1.3.Manfaat Praktikum

1. Mahasiswa dapat memahami konsep pengurutan data.
2. Mahasiswa mampu mengimplementasikan algoritma sorting dalam program java.
3. Mahasiswa dapat membandingkan cara kerja beberapa metode sorting.
4. Mahasiswa dapat melatih logika pemrograman dan pemecahan masalah.
5. Mahasiswa memahami pembuatan GUI sederhana untuk visualisasi algoritma sortin

BAB II

PEMBAHASAN

2.1. Deskripsi Materi Sorting

Sorting merupakan penyusunan data ke dalam sebuah urutan tertentu. Sorting banyak sekali digunakan untuk mengolah data karena dapat mempermudah pencarian dan pengelompokan data.

Berikut beberapa jenis sorting yang dipelajari pada praktikum ini:

a. Insertion Sort

Metode yang bekerja dengan cara menyisipkan elemen ke posisi yang sesuai pada bagian array yang sudah terurut.

b. Selection Sort

Metode sorting yang bekerja dengan mencari nilai terkecil pada array lalu menukarnya dengan elemen pada posisi awal.

c. Bubble Sort

Metode sorting yang bekerja dengan membandingkan dua elemen yang berdekatan lalu menukarnya apabila urutannya salah.

Jenis – jenis tersebut termasuk algoritma sorting sederhana yang mudah dipahami dan diimplementasikan.

2.2. InsertionSort_2511532005

```

1 package pekan7_2511532005;
2
3 public class InsertionSort_2511532005 {
4     public static void insertionSort_2511532005 (int[] arr) {
5         int n = arr.length;
6         for (int i_2005 = 1; i_2005 < n; i_2005++) {
7             int key_2005 = arr[i_2005];
8             int j_2005 = i_2005 - 1;
9             while (j_2005 >= 0 && arr[j_2005] > key_2005) {
10                 arr[j_2005 + 1] = arr[j_2005];
11                 j_2005--;
12             }
13             arr[j_2005 + 1] = key_2005;
14         }
15     }
16     public static void main(String[] args) {
17         int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
18         int n = arr.length;
19         System.out.printf("array yang belum terurut: \n");
20         for (int i_2005 = 0; i_2005 < n; i_2005++)
21             System.out.print(arr[i_2005] + " ");
22         System.out.println("");
23         insertionSort_2511532005(arr);
24         System.out.printf("array yang terurut : \n");
25
26         for (int i_2005 = 0; i_2005 < n; i_2005++)
27             System.out.print(arr[i_2005] + " ");
28         System.out.println(" ");
29     }
30 }
31 }

```

Gambar 2.1. Kode Program InsertionSort_2511532005

Kode dan fungsi nya :

- `int n = arr.length;`
Digunakan untuk mengambil jumlah elemen array.
- `for (int i_2005 = 1; i_2005 < n; i_2005++)`
Perulangan dimulai dari indeks ke-1 karena elemen pertama dianggap sudah terurut.
- `int key_2005 = arr[i_2005];`
Menyimpan nilai yang akan disisipkan.
- `while (j_2005 >= 0 && arr[j_2005] > key_2005)`
Membandingkan elemen sebelumnya dengan nilai key.
- `arr[j_2005 + 1] = arr[j_2005];`
Menggeser elemen ke kanan jika lebih besar dari key.
- `arr[j_2005 + 1] = key_2005;`
Menempatkan key pada posisi yang sesuai.
- Method `main()` digunakan untuk menampilkan array sebelum dan sesudah diurutkan.

2.3. SelectionSort_2511532005.

```

1 package pekan7_2511532005;
2
3 public class SelectionSort_2511532005 {
4     public static void selectionSort_2511532005(int[] arr) {
5         int n = arr.length;
6         for (int i_2005 = 0; i_2005 < n; i_2005++) {
7             int minIndex_2005 = i_2005;
8             for (int j_2005 = i_2005 + 1; j_2005 < n; j_2005++) {
9                 if (arr[j_2005] < arr[minIndex_2005]) {
10                     minIndex_2005 = j_2005;
11                 }
12             }
13             int temp = arr[i_2005];
14             arr[i_2005] = arr[minIndex_2005];
15             arr[minIndex_2005] = temp;
16         }
17     }
18
19     public static void main(String[] args) {
20         int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
21         int n = arr.length;
22         System.out.printf("array yang belum terurut: \n");
23         for (int i_2005 = 0; i_2005 < n; i_2005++)
24             System.out.print(arr[i_2005] + " ");
25         System.out.println("");
26         selectionSort_2511532005(arr);
27         System.out.printf("array yang terurut : \n");
28
29         for (int i_2005 = 0; i_2005 < n; i_2005++)
30             System.out.print(arr[i_2005] + " ");
31         System.out.println(" ");
32     }
33 }
34

```

Gambar 2.2. Kode Program SelectionSort_2511532005.

Kode program dan fungsinya :

- `int minIndex_2005 = i_2005;`
Menyimpan indeks nilai terkecil sementara.
- `for (int j_2005 = i_2005 + 1; j_2005 < n; j_2005++)`
Digunakan untuk mencari nilai terkecil pada array.
- `if (arr[j_2005] < arr[minIndex_2005])`
Membandingkan elemen array.
- `minIndex_2005 = j_2005;`
Menyimpan indeks nilai terkecil baru.
- Proses pertukaran data dilakukan menggunakan variabel temp.
- Method main() menampilkan data sebelum dan sesudah sorting.

2.4. BubbleSort_2511532005

```

1  package pekan7_2511532005;
2
3  public class BubbleSort_2511532005 {
4      public static void bubbleSort_2511532005(int[] arr) {
5          int n = arr.length;
6          for (int i_2005 = 0; i_2005 < n; i_2005++) {
7              for (int j_2005 = 0; j_2005 < n - i_2005 - 1; j_2005++) {
8                  if (arr[j_2005] > arr[j_2005 + 1]) {
9                      int temp = arr[j_2005];
10                     arr[j_2005] = arr[j_2005 + 1];
11                     arr[j_2005 + 1] = temp;
12                     // System.out.println("data:" + arr[j_2005]+" "+arr[j_2005+1]);
13                 }
14             }
15         }
16     }
17     public static void main(String[] args) {
18         int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
19         int n = arr.length;
20         System.out.printf("array yang belum terurut: \n");
21         for (int i_2005 = 0; i_2005 < n; i_2005++)
22             System.out.print(arr[i_2005] + " ");
23         System.out.println("");
24         //minMaxSelectionSort(arr,n);
25
26         bubbleSort_2511532005(arr);
27         System.out.printf("array yang terurut menggunakan BubbleSort:\n");
28         for (int i_2005 = 0; i_2005 < n; i_2005++)
29             System.out.print(arr[i_2005] + " ");
30         System.out.println(" ");
31     }
32 }
33
34

```

Gambar 2.3. Kode Program BubbleSort_2511532005

Kode program dan fungsinya:

- perulangan pertama digunakan untuk menentukan jumlah proses sorting.
- Perulangan kedua digunakan untuk membandingkan elemen yang berdekatan.
- if (arr[j_2005] > arr[j_2005 + 1])
Mengecek apakah elemen kiri lebih besar dari elemen kanan.
- Jika kondisi benar, maka dilakukan pertukaran data menggunakan variabel temp.
- Setelah seluruh proses selesai, data akan terurut secara ascending.

2.5. InsertionGUI_2511532005

```

1 package pekan7_2511532005;
2
3 import java.awt.BorderLayout;
4
22 public class InsertionGUI_2511532005 extends JFrame {
23
24     private static final long serialVersionUID = 1L;
25     private JPanel contentPane;
26     private int[] array_2005;
27     private JLabel[] labelArray_2005;
28     private JButton stepButton_2005, resetButton_2005, setButton_2005;
29     private JTextField inputField_2005;
30     private JPanel panelArray_2005;
31     private JTextArea stepArea_2005;
32
33     private int i_2005 = 1, j_2005;
34     private boolean sorting_2005 = false;
35     private int stepCount_2005 = 1;
36
37
38
39 /**
40  * Create the frame.
41  */
42 public InsertionGUI_2511532005() {
43     setTitle("Insertion Sort Langkah per Langkah");
44     setSize(750, 400);
45     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46     setLocationRelativeTo(null);
47     setLayout(new BorderLayout());
48
49     //Panel Input
50     JPanel inputPanel = new JPanel(new FlowLayout());
51     inputField_2005 = new JTextField(30);
52     setButton_2005 = new JButton("Set Array");
53     inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma): "));
54     inputPanel.add(inputField_2005);
55     inputPanel.add(setButton_2005);
56
57     //Panel array visual
58     panelArray_2005 = new JPanel();
59     panelArray_2005.setLayout(new FlowLayout());
60
61     //panel kontrol
62     JPanel controlPanel = new JPanel();
63     setButton_2005 = new JButton("Langkah selanjutnya");
64     resetButton_2005 = new JButton("Reset");
65     stepButton_2005 = new JButton("Reset");
66     controlPanel.add(stepButton_2005);
67     controlPanel.add(resetButton_2005);
68
69     //area teks untuk log langkah-langkah
70     stepArea_2005 = new JTextArea(8, 60);
71     stepArea_2005.setEditable(false);
72     stepArea_2005.setFont(new Font("Monospace", Font.PLAIN, 14));
73     JScrollPane scrollPane = new JScrollPane(stepArea_2005);
74
75     //tambahkan panel ke frame
76     add(inputPanel, BorderLayout.NORTH);
77     add(panelArray_2005, BorderLayout.CENTER);
78     add(controlPanel, BorderLayout.SOUTH);
79     add(scrollPane, BorderLayout.EAST);
80
81     // Event Set Array
82     setButton_2005.addActionListener(e -> setArrayFromInput_2511532005());
83
84     // Event Langkah Selanjutnya
85     stepButton_2005.addActionListener(e -> performStep());
86
87     // Event Reset
88     resetButton_2005.addActionListener(e -> reset());
89
90     private void setArrayFromInput_2511532005() {
91         String text = inputField_2005.getText().trim();
92         if (text.isEmpty()) return;
93         String[] parts = text.split(",");
94         array_2005 = new int[parts.length];
95         try {
96             for (int k = 0; k < parts.length; k++) {
97                 array_2005[k] = Integer.parseInt(parts[k].trim());
98             }
99         } catch (NumberFormatException e) {
100             JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan dengan koma", "Error", JOptionPane.ERROR_MESSAGE);
101             return;
102         }
103         i_2005 = 1;
104         stepCount_2005 = 1;
105         sorting_2005 = true;
106         stepButton_2005.setEnabled(true);
107         stepArea_2005.setText("");
108         panelArray_2005.removeAll();
109         labelArray_2005 = new JLabel[array_2005.length];
110         for (int k = 0; k < array_2005.length; k++) {
111             labelArray_2005[k] = new JLabel(String.valueOf(array_2005[k]));
112             labelArray_2005[k].setFont(new Font("Arial", Font.BOLD, 20));
113             labelArray_2005[k].setBorder(BorderFactory.createInsetBorder(Color.BLACK));
114             labelArray_2005[k].setPreferredSize(new Dimension(50, 30));
115             labelArray_2005[k].setHorizontalAlignment(SwingConstants.CENTER);
116             panelArray_2005.add(labelArray_2005[k]);
117         }
118         panelArray_2005.revalidate();
119         panelArray_2005.repaint();
120     }
121
122     private void performStep() {
123         if (i_2005 < array_2005.length && sorting_2005) {
124             int key = array_2005[i_2005];
125             j_2005 = i_2005 - 1;
126
127             StringBuilder stepLog = new StringBuilder();
128             stepLog.append("Langkah ")
129                 .append(stepCount_2005)
130                 .append(": Memasukkan ")
131                 .append(key)
132                 .append("\n");
133             while (j_2005 >= 0 && array_2005[j_2005] > key) {

```

```

137         array_2005[j_2005 + 1] = array_2005[j_2005];
138         j_2005--;
139     }
140
141     array_2005[j_2005 + 1] = key;
142
143     updateLabels();
144
145     stepLog.append("Hasil: ")
146             .append(arrayToString(array_2005))
147             .append("\n\n");
148
149     stepArea_2005.append(stepLog.toString());
150
151     i_2005++;
152     stepCount_2005++;
153
154     if (i_2005 == array_2005.length) {
155
156         sorting_2005 = false;
157         stepButton_2005.setEnabled(false);
158
159         JOptionPane.showMessageDialog(
160             this,
161             "Sorting selesai!"
162         );
163     }
164 }
165
166 private void updateLabels() {
167
168     for (int k = 0; k < array_2005.length; k++) {
169
170         labelArray_2005[k].setText(
171             String.valueOf(array_2005[k])
172         );
173     }
174 }
175

```

```

175
176 private void reset() {
177
178     inputField_2005.setText("");
179
180     panelArray_2005.removeAll();
181     panelArray_2005.revalidate();
182     panelArray_2005.repaint();
183
184     stepArea_2005.setText("");
185
186     stepButton_2005.setEnabled(false);
187
188     sorting_2005 = false;
189
190     i_2005 = 1;
191     stepCount_2005 = 1;
192 }
193
194 private String arrayToString(int[] arr) {
195
196     StringBuilder sb = new StringBuilder();
197
198     for (int k = 0; k < arr.length; k++) {
199
200         sb.append(arr[k]);
201
202         if (k < arr.length - 1) {
203             sb.append(", ");
204         }
205     }
206
207     return sb.toString();
208 }
209
210 public static void main(String[] args) {
211     SwingUtilities.invokeLater(() -> {
212         InsertionGUI_2511532005 gui = new InsertionGUI_2511532005();
213         gui.setVisible(true);
214     });
215 }
216

```

Gambar 2.4. InsertionGUI_2511532005

Fungsi dan komponen Program :

- a. JFrame, digunakan sebagai jendela utama aplikasi GUI.
- b. JTextField, digunakan untuk memasukkan data array yang dipisahkan dengan tanda koma.
- c. JButton, ini memiliki beberapa tombol, yaitu untuk memasukkan array, Langkah selanjutnya, dan reset atau mengulang program.
- d. JPanel, digunakan untuk menampilkan visualisasi array.
- e. JTextArea, digunakan untuk menampilkan log Langkah-langkah sorting.
- f. Method `setArrayFromInput_2511532005()`
Berfungsi membaca input pengguna lalu mengubahnya menjadi array integer.
- g. Method `performStep()`
Menjalankan satu langkah proses Insertion Sort setiap kali tombol ditekan.
- h. Method `updateLabels()`
Memperbarui tampilan array pada GUI setelah proses sorting.
- i. Method `reset()`
Mengembalikan program ke kondisi awal.
- j. Method `arrayToString()`
Mengubah array menjadi string agar mudah ditampilkan pada JTextArea.
- k. Method `main()`
Digunakan untuk menjalankan GUI menggunakan `SwingUtilities.invokeLater()`.

2.6. Analisis Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat diketahui bahwa algoritma sorting seperti Insertion Sort, Selection Sort, dan Bubble Sort memiliki cara kerja yang berbeda dalam mengurutkan data. Insertion Sort bekerja dengan menyisipkan elemen ke posisi yang tepat, Selection Sort

mencari nilai terkecil untuk ditukar ke posisi awal, sedangkan Bubble Sort membandingkan elemen yang berdekatan secara berulang. Dari hasil percobaan, seluruh metode berhasil mengurutkan data secara ascending, namun masing-masing memiliki tingkat efisiensi yang berbeda. Selain itu, penggunaan GUI pada Insertion Sort membantu memvisualisasikan proses pengurutan langkah demi langkah sehingga lebih mudah dipahami.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa proses sorting merupakan teknik penting dalam pengolahan data untuk mengurutkan data secara terstruktur. Pada praktikum ini telah dipelajari tiga metode sorting yaitu Insertion Sort, Selection Sort, dan Bubble Sort yang masing-masing memiliki cara kerja berbeda namun menghasilkan output yang sama yaitu data terurut secara ascending. Selain itu, pembuatan GUI pada Insertion Sort membantu memahami proses pengurutan secara visual dan interaktif sehingga mempermudah pembelajaran algoritma sorting.

3.2. Saran

Pada praktikum selanjutnya, disarankan agar praktikan mencoba menggunakan jumlah data yang lebih besar untuk melihat perbedaan efisiensi masing-masing algoritma sorting. Selain itu, tampilan GUI dapat dikembangkan menjadi lebih menarik dan interaktif agar proses pembelajaran semakin mudah dipahami. Praktikan juga disarankan mempelajari algoritma sorting lainnya seperti Quick Sort dan Merge Sort untuk menambah pemahaman tentang metode pengurutan data yang lebih efisien.

DAFTAR PUSTAKA

- Hanilatifa, Z. N. (2025). *Struktur Data: Sorting*. Retrieved from Medium.com: <https://zahrahani.medium.com/struktur-data-sorting-6a30fe60684f>
- Zipur, A. (n.d.). *#9 Struktur Data : Sorting – Pengertian, Manfaat dan Algoritmanya*. Retrieved from ahmadzipur.com: <https://ahmadzipur.com/9-struktur-data-sorting-pengertian-manfaat-dan-algoritmanya/>

TUGAS PRAKTIKUM 7

1. Perintah Tugas

Program dikembangkan menggunakan Java GUI (Swing) untuk menampilkan proses pengurutan data mahasiswa secara interaktif.

Program menyediakan beberapa fitur utama, yaitu:

- Input data mahasiswa secara manual melalui form GUI.
- Menyimpan data mahasiswa ke dalam array atau ArrayList object.
- Pemilihan algoritma sorting menggunakan ComboBox.
- Menampilkan visualisasi proses sorting langkah demi langkah.
- **Menampilkan hasil akhir pengurutan nama mahasiswa secara ascending (A-Z).**

Perbandingan string dilakukan menggunakan method `compareToIgnoreCase()` agar proses sorting tidak membedakan huruf kapital dan huruf kecil.

2. ADT Mahasiswa

```

1 package pekan7_2511532005;
2
3 public class Mahasiswa_2511532005 {
4     private String nama;
5     private String nim;
6     private String prodi;
7
8     public Mahasiswa_2511532005(String nama, String nim, String prodi) {
9         this.nama = nama;
10        this.nim = nim;
11        this.prodi = prodi;
12    }
13
14    public String getName() { return nama; }
15    public void setName(String nama) { this.nama = nama; }
16    public String getNim() { return nim; }
17    public void setNim(String nim) { this.nim = nim; }
18    public String getProdi() { return prodi; }
19    public void setProdi(String prodi) { this.prodi = prodi; }
20 }

```


Program di atas digunakan untuk membuat class Mahasiswa_2511532005 yang berfungsi menyimpan data mahasiswa berupa nama, NIM, dan program studi.

Penjelasan kode:

- private String nama; , digunakan untuk menyimpan nama mahasiswa.
- private String nim;, digunakan untuk menyimpan NIM mahasiswa.
- private String prodi; digunakan untuk menyimpan program studi mahasiswa.
- public Mahasiswa_2511532005(String nama, String nim, String prodi) merupakan constructor yang digunakan untuk mengisi data mahasiswa saat objek dibuat.
- this.nama = nama; untuk mengisi atribut nama dengan nilai yang dimasukkan.
- this.nim = nim untuk mengisi atribut nim dengan nilai yang dimasukkan.
- this.prodi = prodi untuk mengisi atribut prodi dengan nilai yang dimasukkan.
- getNama(), getNim(), getProdi(): berfungsi untuk mengambil data atribut mahasiswa.
- setNama(), setNim(), setProdi() : Berfungsi untuk mengubah data atribut mahasiswa.

3. Drive Sorting

```

1 package pekan7_2511532005;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8 import java.awt.GridLayout;
9
10 import javax.swing.BorderFactory;
11 import javax.swing.JButton;
12 import javax.swing.JComboBox;
13 import javax.swing.JFrame;
14 import javax.swing.JLabel;
15 import javax.swing.JOptionPane;
16 import javax.swing.JPanel;
17 import javax.swing.JScrollPane;
18 import javax.swing.JTextArea;
19 import javax.swing.JTextField;
20 import javax.swing.SwingConstants;
21 import javax.swing.SwingUtilities;
22 import java.util.ArrayList;
23
24 public class SortingGUI_2511532005 extends JFrame {
25
26     private static final long serialVersionUID = 1L;
27
28     // Komponen GUI
29     private JTextField tfNama, tfNim, tfProdi;
30     private JLabel[] labelArray_2005;
31     private JButton btnTambah_2005, btnMulai_2005, btnReset_2005;
32     private JComboBox<String> cbAlgoritma;
33     private JPanel panelArray_2005;
34     private JTextArea stepArea_2005;
35
36     // Data mahasiswa
37     private ArrayList<Mahasiswa_2511532005> listMahasiswa;
38
39     private ArrayList<Mahasiswa_2511532005> dataSort;
40
41     // Variabel sorting
42     private int i_2005 = 1, j_2005;
43     private boolean sorting_2005 = false;
44     private int stepCount_2005 = 1;
45     private String algoritma_2005;
46
47     public SortingGUI_2511532005() {
48         setTitle("Sorting Nama Mahasiswa - 2511532005");
49         setSize(950, 550);
50         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
51         setLocationRelativeTo(null);
52         setLayout(new BorderLayout());
53
54         listMahasiswa = new ArrayList<>();
55         dataSort = new ArrayList<>();
56
57         inisialisasiKomponen();
58         eventListeners();
59
60     private void inisialisasiKomponen() {
61         // Panel Input (atas - dengan GridLayout agar lebih jelas)
62         JPanel inputPanel = new JPanel(new GridLayout(2, 4, 5, 5));
63         inputPanel.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));
64         inputPanel.setPreferredSize(new Dimension(80, 80));
65
66         // Label dan TextField
67         tfNama = new JTextField(15);
68         tfNim = new JTextField(10);
69         tfProdi = new JTextField(10);
70         btnTambah_2005 = new JButton("+ Tambah Data");
71
72         // PASTIKAN EDITABLE
73         tfNama.setEditable(true);
74         tfNim.setEditable(true);

```

```

75         tfProdi.setEditable(true);
76
77         // Set font agar lebih jelas
78         tfNama.setFont(new Font("Arial", Font.PLAIN, 14));
79         tfNim.setFont(new Font("Arial", Font.PLAIN, 14));
80         tfProdi.setFont(new Font("Arial", Font.PLAIN, 14));
81
82         inputPanel.add(new JLabel("Nama:"));
83         inputPanel.add(tfNama);
84         inputPanel.add(new JLabel("NIM:"));
85         inputPanel.add(tfNim);
86         inputPanel.add(new JLabel("Prodi:"));
87         inputPanel.add(tfProdi);
88         inputPanel.add(btnTambah_2005);
89
90         // Panel Sorting (bagian tengah atas)
91         JPanel sortingPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
92         sortingPanel.setBorder(BorderFactory.createTitledBorder("Pilihan Algoritma"));
93         sortingPanel.setPreferredSize(new Dimension(0, 70));
94
95         cbAlgoritma = new JComboBox<>(new String[]{"Insertion Sort", "Selection Sort", "Bubble Sort"});
96         btnMulai_2005 = new JButton("Mulai Sorting");
97         btnReset_2005 = new JButton("Reset");
98         btnMulai_2005.setEnabled(false);
99
100        sortingPanel.add(new JLabel("Algoritma:"));
101        sortingPanel.add(cbAlgoritma);
102        sortingPanel.add(btnMulai_2005);
103        sortingPanel.add(btnReset_2005);
104
105        // Panel Visual Array (tengah)
106        panelArray_2005 = new JPanel(new FlowLayout());
107        panelArray_2005.setBorder(BorderFactory.createTitledBorder("Visualisasi Data"));
108
109        // Panel Langkah (kanan)
110        stepArea_2005 = new JTextArea(15, 45);
111        stepArea_2005.setEditable(false);
112
113        stepArea_2005.setFont(new Font("Monospace", Font.PLAIN, 13));
114        JScrollPane scrollPane = new JScrollPane(stepArea_2005);
115        scrollPane.setBorder(BorderFactory.createTitledBorder("Proses Sorting"));
116
117        // Panel Utama
118        JPanel centerPanel = new JPanel(new BorderLayout());
119        centerPanel.add(sortingPanel, BorderLayout.NORTH);
120        centerPanel.add(panelArray_2005, BorderLayout.CENTER);
121
122        // Tambahkan ke frame
123        add(inputPanel, BorderLayout.NORTH);
124        add(centerPanel, BorderLayout.CENTER);
125        add(scrollPane, BorderLayout.EAST);
126    }
127
128    private void eventListeners() {
129        btnTambah_2005.addActionListener(e -> tambahData());
130        btnMulai_2005.addActionListener(e -> mulaiSorting());
131        btnReset_2005.addActionListener(e -> reset());
132    }
133
134    private void tambahData() {
135        String nama = tfNama.getText().trim();
136        String nim = tfNim.getText().trim();
137        String prodi = tfProdi.getText().trim();
138
139        if (nama.isEmpty() || nim.isEmpty() || prodi.isEmpty()) {
140            JOptionPane.showMessageDialog(this, "Mohon isi semua data!\nNama, NIM, dan Prodi wajib diisi",
141                "Peringatan", JOptionPane.WARNING_MESSAGE);
142            return;
143        }
144
145        // Debug: cek nilai
146        System.out.println("Nama: " + nama);
147        System.out.println("NIM: " + nim);
148        System.out.println("Prodi: " + prodi);
149
150        Mahasiswa mhs = new Mahasiswa(2511532005, nama, nim, prodi);
151        listMahasiswa.add(mhs);
152
153        System.out.println("Jumlah data: " + listMahasiswa.size());
154
155        // Update visual array
156        updateVisualArray();
157
158        // Clear input
159        tfNama.setText("");
160        tfNim.setText("");
161        tfProdi.setText("");
162        tfNama.requestFocus();
163
164        btnMulai_2005.setEnabled(true);
165
166        JOptionPane.showMessageDialog(this, "Data berhasil ditambahkan!\nTotal: " + listMahasiswa.size() + " mahasiswa");
167    }
168
169    private void updateVisualArray() {
170        panelArray_2005.removeAll();
171        labelArray_2005 = new JLabel[listMahasiswa.size()];
172
173        for (int k = 0; k < listMahasiswa.size(); k++) {
174            String display = listMahasiswa.get(k).getNama() + "\n" + listMahasiswa.get(k).getNim();
175            labelArray_2005[k] = new JLabel("<html><center>" + display.replace("\n", "<br>") + "</center></html>");
176            labelArray_2005[k].setFont(new Font("Arial", Font.BOLD, 12));
177            labelArray_2005[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
178            labelArray_2005[k].setPreferredSize(new Dimension(90, 55));
179            labelArray_2005[k].setHorizontalAlignment(SwingConstants.CENTER);
180            labelArray_2005[k].setOpaque(true);
181            labelArray_2005[k].setBackground(Color.WHITE);
182
183            panelArray_2005.add(labelArray_2005[k]);
184        }
185        panelArray_2005.revalidate();

```

```

186     panelArray_2005.repaint();
187 }
188
189 private void mulaiSorting() {
190     if (listMahasiswa.isEmpty()) {
191         JOptionPane.showMessageDialog(this, "Data mahasiswa kosong!\nSilakan tambah data terlebih dahulu",
192             "Peringatan", JOptionPane.WARNING_MESSAGE);
193         return;
194     }
195
196     // Buat salinan data
197     dataSort.clear();
198     for (Mahasiswa_2511532005 mhs : listMahasiswa) {
199         dataSort.add(new Mahasiswa_2511532005(mhs.getNama(), mhs.getNim(), mhs.getProdi()));
200     }
201
202     algoritma_2005 = (String) cbAlgoritma.getSelectedItem();
203     stepArea_2005.setText("");
204
205     i_2005 = 1;
206     j_2005 = 0;
207     sorting_2005 = true;
208     stepCount_2005 = 1;
209
210     if (algoritma_2005.equals("Insertion Sort")) {
211         insertionSortStep();
212     } else if (algoritma_2005.equals("Selection Sort")) {
213         selectionSortStep();
214     } else if (algoritma_2005.equals("Bubble Sort")) {
215         bubbleSortStep();
216     }
217 }
218
219 private void insertionSortStep() {
220     StringBuilder stepLog = new StringBuilder();
221     stepLog.append("=== INSERTION SORT ===\n\n");
222     stepLog.append("Data awal: ").append(arrayToString(dataSort)).append("\n\n");

```

```

223
224     for (int i = 1; i < dataSort.size(); i++) {
225         Mahasiswa_2511532005 key = dataSort.get(i);
226         int j = i - 1;
227
228         stepLog.append("Langkah ").append(i).append(": ");
229         stepLog.append(arrayToString(dataSort));
230
231         while (j >= 0 && dataSort.get(j).getNama().compareToIgnoreCase(key.getNama()) > 0) {
232             dataSort.set(j + 1, dataSort.get(j));
233             j--;
234         }
235
236         dataSort.set(j + 1, key);
237         stepLog.append(" -> ").append(arrayToString(dataSort)).append("\n\n");
238     }
239
240     stepLog.append("=== HASIL AKHIR ===\n\n");
241     stepLog.append(arrayToString(dataSort)).append("\n\n");
242
243     stepArea_2005.setText(stepLog.toString());
244     updateVisualArrayFinal();
245
246     sorting_2005 = false;
247     JOptionPane.showMessageDialog(this, "Sorting selesai!");
248 }
249
250 private void selectionSortStep() {
251     StringBuilder stepLog = new StringBuilder();
252     stepLog.append("=== SELECTION SORT ===\n\n");
253     stepLog.append("Data awal: ").append(arrayToString(dataSort)).append("\n\n");
254
255     for (int i = 0; i < dataSort.size() - 1; i++) {
256         int minIdx = i;
257
258         for (int j = i + 1; j < dataSort.size(); j++) {
259             if (dataSort.get(i).getNama().compareToIgnoreCase(dataSort.get(minIdx).getNama()) < 0) {

```

```

260                 minIdx = j;
261             }
262         }
263
264         if (minIdx != i) {
265             stepLog.append("Pass ").append(i + 1).append(": ");
266             stepLog.append(arrayToString(dataSort));
267
268             Mahasiswa_2511532005 temp = dataSort.get(i);
269             dataSort.set(i, dataSort.get(minIdx));
270             dataSort.set(minIdx, temp);
271
272             stepLog.append(" -> ").append(arrayToString(dataSort)).append("\n\n");
273         } else {
274             stepLog.append("Pass ").append(i + 1).append(": ");
275             stepLog.append(arrayToString(dataSort)).append(" (tidak perlu ditukar)\n\n");
276         }
277     }
278
279     stepLog.append("\n=== HASIL AKHIR ===\n\n");
280     stepLog.append(arrayToString(dataSort)).append("\n\n");
281
282     stepArea_2005.setText(stepLog.toString());
283     updateVisualArrayFinal();
284
285     sorting_2005 = false;
286     JOptionPane.showMessageDialog(this, "Sorting selesai!");
287 }
288
289 private void bubbleSortStep() {
290     StringBuilder stepLog = new StringBuilder();
291     stepLog.append("=== BUBBLE SORT ===\n\n");
292     stepLog.append("Data awal: ").append(arrayToString(dataSort)).append("\n\n");
293
294     int n = dataSort.size();
295     boolean adaPertukaran;
296

```

```

297●     for (int pass = 1; pass < n; pass++) {
298         adaPenukaran = false;
299         stepLog.append("Pass ").append(pass).append(": ");
300
301●         for (int j = 0; j < n - pass; j++) {
302●             if (dataSort.get(j).getNama().compareToIgnoreCase(dataSort.get(j + 1).getNama()) > 0) {
303                 Mahasiswa_2511532005 temp = dataSort.get(j);
304                 dataSort.set(j, dataSort.get(j + 1));
305                 dataSort.set(j + 1, temp);
306                 adaPenukaran = true;
307             }
308         }
309
310         stepLog.append(arrayToString(dataSort));
311
312●         if (!adaPenukaran) {
313             stepLog.append(" (tidak ada penukaran)\n");
314             break;
315         }
316         stepLog.append("\n");
317     }
318
319     stepLog.append("\n== HASIL AKHIR ==\n");
320     stepLog.append(arrayToString(dataSort)).append("\n");
321
322     stepArea_2005.setText(stepLog.toString());
323     updateVisualArrayFinal();
324
325     sorting_2005 = false;
326     JOptionPane.showMessageDialog(this, "Sorting selesai!");
327 }
328
329● private void updateVisualArrayFinal() {
330     panelArray_2005.removeAll();
331     labelArray_2005 = new JLabel[dataSort.size()];
332
333     for (int k = 0; k < dataSort.size(); k++) {
334
335         String display = dataSort.get(k).getNama() + "\n" + dataSort.get(k).getNim();
336         labelArray_2005[k] = new JLabel("<html><center>" + display.replace("\n", "<br>" + "</center></html>");
337         labelArray_2005[k].setFont(new Font("Arial", Font.BOLD, 12));
338         labelArray_2005[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
339         labelArray_2005[k].setPreferredSize(new Dimension(90, 55));
340         labelArray_2005[k].setHorizontalAlignment(SwingConstants.CENTER);
341         labelArray_2005[k].setOpaque(true);
342         labelArray_2005[k].setBackground(new Color(200, 255, 200));
343
344         panelArray_2005.add(labelArray_2005[k]);
345     }
346
347     panelArray_2005.revalidate();
348     panelArray_2005.repaint();
349 }
350
351 private void reset() {
352     int confirm = JOptionPane.showConfirmDialog(this,
353         "Yakin ingin mereset semua data?", "Konfirmasi",
354         JOptionPane.YES_NO_OPTION);
355
356     if (confirm == JOptionPane.YES_OPTION) {
357         tfNama.setText("");
358         tfNim.setText("");
359         tfProdi.setText("");
360
361         panelArray_2005.removeAll();
362         panelArray_2005.revalidate();
363         panelArray_2005.repaint();
364
365         stepArea_2005.setText("");
366         listMahasiswa.clear();
367         dataSort.clear();
368
369         btnMulai_2005.setEnabled(false);
370         sorting_2005 = false;
371         i_2005 = 1;
372     }
373 }
374
375 private String arrayToString(ArrayList<Mahasiswa_2511532005> arr) {
376     StringBuilder sb = new StringBuilder();
377     sb.append("[");
378     for (int k = 0; k < arr.size(); k++) {
379         sb.append(arr.get(k).getNama());
380         if (k < arr.size() - 1) {
381             sb.append(", ");
382         }
383     }
384     sb.append("]");
385     return sb.toString();
386 }
387
388 public static void main(String[] args) {
389     SwingUtilities.invokeLater(() -> {
390         SortingGUI_2511532005 gui = new SortingGUI_2511532005();
391         gui.setVisible(true);
392     });
393 }
394 }

```

Program SortingGUI_2511532005 digunakan untuk membuat aplikasi GUI Java Swing yang berfungsi mengurutkan data mahasiswa berdasarkan nama menggunakan algoritma Insertion Sort, Selection Sort, dan Bubble Sort.

Penjelasan kode:

- extends JFrame, digunakan agar class dapat menjadi jendela GUI.
- JTextField tfNama, tfNim, tfProdi; digunakan untuk input data mahasiswa berupa nama, NIM, dan program studi.
- JButton btnTambah_2005, btnMulai_2005, btnReset_2005; tombol untuk menambah data, memulai sorting, dan reset data.
- JComboBox<String> cbAlgoritma; digunakan untuk memilih algoritma sorting.
- ArrayList<Mahasiswa_2511532005> listMahasiswa; menyimpan data mahasiswa yang dimasukkan pengguna.
- inisialisasiKomponen() berfungsi membuat dan mengatur tampilan GUI.
- eventListeners() mengatur aksi pada tombol.
- tambahData() digunakan untuk menambahkan data mahasiswa ke dalam list.
- updateVisualArray() menampilkan data mahasiswa pada panel visual.
- mulaiSorting() memulai proses sorting sesuai algoritma yang dipilih.
- insertionSortStep() mengurutkan data menggunakan metode Insertion Sort.
- selectionSortStep() mengurutkan data menggunakan metode Selection Sort.
- bubbleSortStep() mengurutkan data menggunakan metode Bubble Sort.
- updateVisualArrayFinal() menampilkan hasil akhir sorting pada GUI.
- reset() menghapus seluruh data dan mengembalikan program ke kondisi awal.
- arrayToString() mengubah data array menjadi string agar mudah ditampilkan.
- main() digunakan untuk menjalankan aplikasi GUI.

Program ini membantu pengguna memahami proses sorting data mahasiswa secara visual dan interaktif menggunakan Java Swing.

4. Output

Sorting Nama Mahasiswa - 2511532005

Input Data Mahasiswa

Nama: NIM:

Prodi: [+ Tambah Data](#)

Pilihan Algoritma

Algoritma: **Bubble Sort** [Mulai Sorting](#) [Reset](#)

Visualisasi Data

caca 2511532015	khairunnisa M 2511532005	nabila 2511531007	rayya 2511532007
--------------------	-----------------------------	----------------------	---------------------

Proses Sorting

=== BUBBLE SORT ===

Data awal: [khairunnisa M, rayya, nabila, caca]

Pass 1: [khairunnisa M, nabila, caca, rayya]
 Pass 2: [khairunnisa M, caca, nabila, rayya]
 Pass 3: [caca, khairunnisa M, nabila, rayya]

=== HASIL AKHIR ===
 [caca, khairunnisa M, nabila, rayya]

Sorting Nama Mahasiswa - 2511532005

Input Data Mahasiswa

Nama: NIM:

Prodi: [+ Tambah Data](#)

Pilihan Algoritma

Algoritma: **Selection Sort** [Mulai Sorting](#) [Reset](#)

Visualisasi Data

caca 2511532015	khairunnisa M 2511532005	nabila 2511531007	rayya 2511532007
--------------------	-----------------------------	----------------------	---------------------

Proses Sorting

=== SELECTION SORT ===

Data awal: [khairunnisa M, rayya, nabila, caca]

Pass 1: [khairunnisa M, rayya, nabila, caca] -> [caca, rayya, nabila, khairunnisa M]
 Pass 2: [caca, rayya, nabila, khairunnisa M] -> [caca, khairunnisa M, nabila, rayya]
 Pass 3: [caca, khairunnisa M, nabila, rayya] (tidak perlu ditukar)

=== HASIL AKHIR ===
 [caca, khairunnisa M, nabila, rayya]

Sorting Nama Mahasiswa - 2511532005

Input Data Mahasiswa

Nama: NIM:

Prodi: [+ Tambah Data](#)

Pilihan Algoritma

Algoritma: **Insertion Sort** [Mulai Sorting](#) [Reset](#)

Visualisasi Data

caca 2511532015	khairunnisa M 2511532005	nabila 2511531007	rayya 2511532007
--------------------	-----------------------------	----------------------	---------------------

Proses Sorting

=== SELECTION SORT ===

Data awal: [khairunnisa M, rayya, nabila, caca]

Pass 1: [khairunnisa M, rayya, nabila, caca] -> [caca, rayya, nabila, khairunnisa M]
 Pass 2: [caca, rayya, nabila, khairunnisa M] -> [caca, khairunnisa M, nabila, rayya]
 Pass 3: [caca, khairunnisa M, nabila, rayya] (tidak perlu ditukar)

=== HASIL AKHIR ===
 [caca, khairunnisa M, nabila, rayya]